

AI Agent 時代的 SRE

讓 Claude 成為你的 On-Call 夥伴

Barry Lo · Trend Micro SRE

2026-04-23 · iThome Observability Day

| 我是 Barry

SRE @ Trend Micro

銀行業 → 資安業

5 年

我的日常

On-call 24/7 · 365 days
10 Backends · 6 Clouds

帶著筆電一起去過...

環球影城

迪士尼

看極光

“

Opsgenie 打給我的次數，
比我家人還多。

SECTION 2

你有沒有被 Alert
淹沒過？

凌晨 02:15

手機震動——Opsgenie 大響

系統炸了

SLA 掉了

我每次要做的事

#	動作	時間
0	愣 5 秒 + Teams 回報	5 min
1	找電腦 + 連 VPN	10 min
2	Dashboard / Monitor 定位	15 min
3	Teams 拉 RD 進群組	5 min

(接續)

#	動作	時間
4	檢查 k8s pods / deployment	10 min
5	OpenSearch 找 log	20 min
6	提出解方 + 執行	30 min
7	Confluence 寫報告	1 hr

加起來——

2+ 小時

大腦燒完，終於可以去睡覺了

“

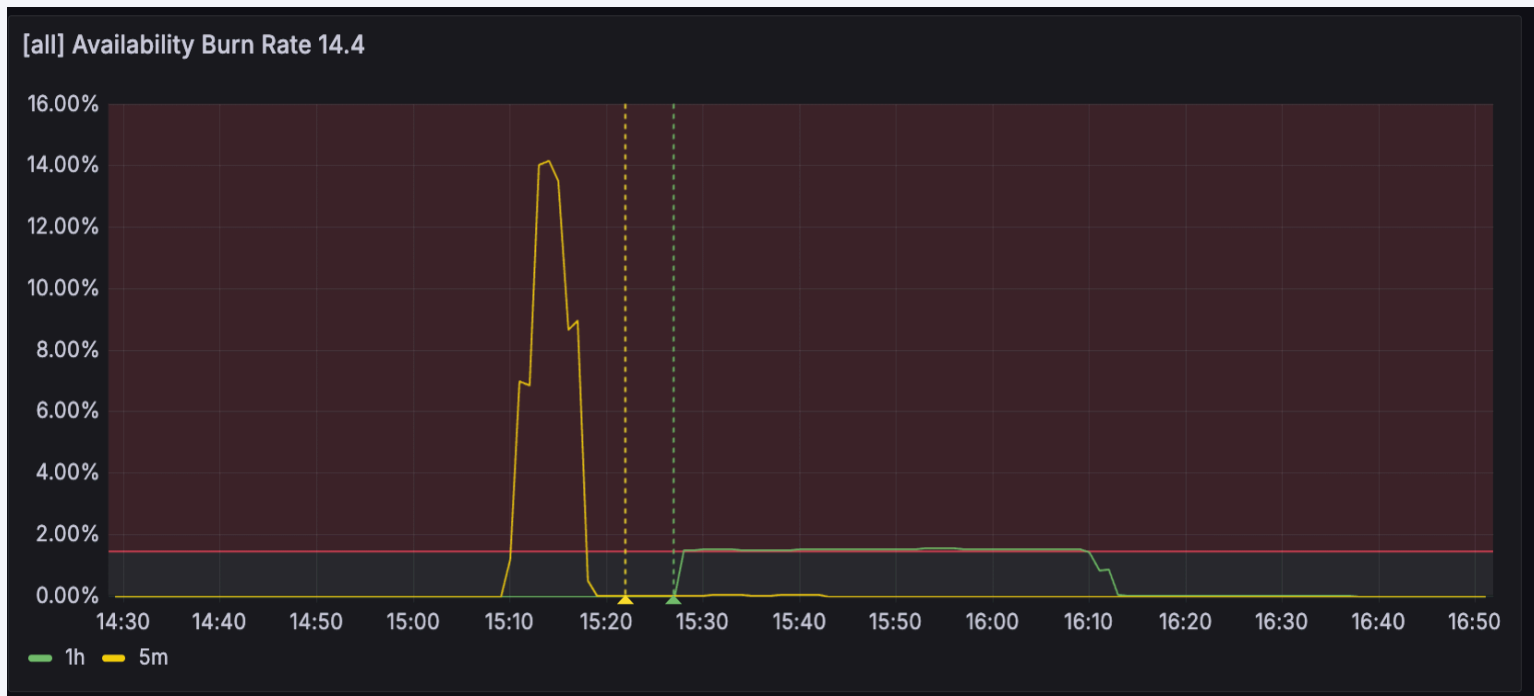
這不是偶爾
這是每次

SECTION 3

看看我現在怎麼做

一個真實 incident 走一遍

某個下午，Opsgenie 大響



SLO Burn Rate 衝破閾值

| 我打了什麼？

```
$ claude
```

xxx domain 5xx error，開啟調查

- 沒告訴它去哪查
- 沒教它用什麼工具
- 沒給 correlation ID

AI 做的第一件事——讀 Skill

● [redacted]-troubleshooter-fast skill

● Thought for 3s

● 解析告警：Application 服務 → [redacted]-application index，時間窗 15:10-15:15 UTC+8 加 ±30 分鐘緩衝 → UTC 06:40-07:45。先載入 OpenSearch MsearchTool。

● Thought for 3s

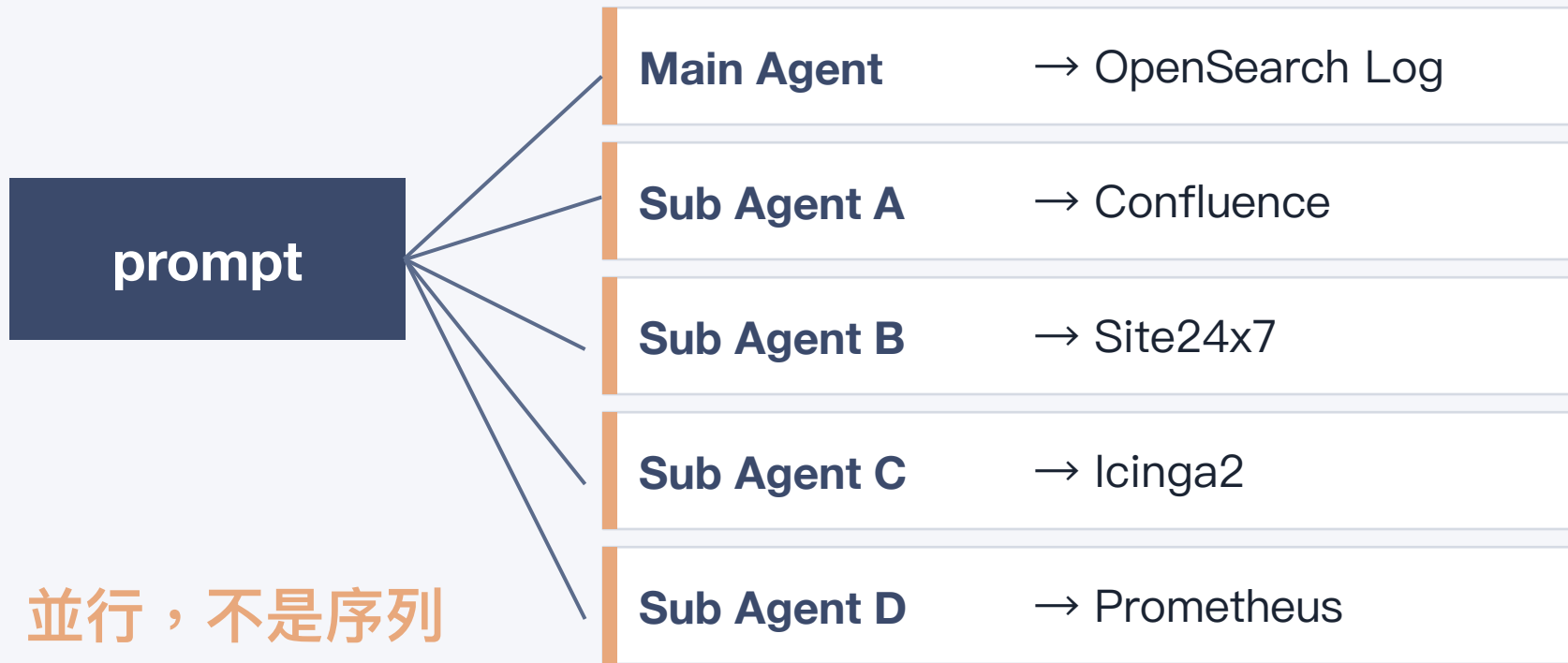
● 查詢 [redacted]-application access log + app log，時間窗 2026-04-07 06:40-07:45 UTC (= 14:40-15:45 UTC+8)。

● Opensearch-mcp-server [MsearchTool]

```
OUT {"took": 7770, "responses": [{"took": 1701, "timed_out": false, "_shards": {"total": 97, "successful": 97, "skipped": 0, "failed": 0}, "hits": {"total": {"value": 10000, "relation": "gte"}, "max_score": null, "hits": []}, "aggregations": {"by_status": {"doc_count_error_upper_bound": 0, "sum_other_doc_count": 0, "buckets":
```

沒有 Skill，AI 可能會走得比人類還慢

然後它做了一件人類做不到的事



| 同一件事，兩種速度

HUMAN · 序列

20 min

一個工具接一個查

AI · 並行

5 min

4 個 agent 同時跑

Sub-agent 帶回來的東西

Update Todos

- ✓ Phase 0: Spawn 4 parallel sub-agents (Confluence/Prom/S247/icinga)
- ✓ Phase 1: OpenSearch deep dive — verify duration=0, pod distribution, profile-service 5xx
- ✓ Phase 2: Aggregate findings + write investigation report

調查完成。報告已寫入 [20260407]_503_pod_restart.md

核心結論

根因: Pod OOM Kill (Exit 137)，觸發器為 Service DB transaction 異常
用戶三個假設驗證結果:

假設	驗證
15:11-15:15 pod 疑似重啟	✓ 正確 — 7 個 pod OOM Kill (dv6hb/bgk2n/j68kk/99b6v/kzcl5/x9tng/dxpgw)，全在同一 ReplicaSet 5c569b8c94
503 duration=0ms	✓ 正確 — 10/10 sample 都是 duration=0，且 correlation_id=null，是 Envoy Connection_refused (基礎架構層)
非應用層錯誤	✓ 部分正確 — 503 本身是 infra 層 (Envoy)，但觸發器是應用層 (goroutine pile-up → OOM)

重要警告: 33 天內第 4 次完全相同事件 (3/5 → 3/10 → 3/23 → 4/7)。Confluence 同事件頁面 已由 RD 用 kubectl 確認 OOM Kill。

Action Items: RD PR#128 / PR#127 已 merged 但未部署，目標 4/30 前 live；SRE OOM Kill (自 3/10 起)。建議升級到 incident review 層級追蹤。

「這是

第 4 次

OOM Kill了。」

AI 把歷史發現，直接寫進報告標題

- 沒卡住，資料已足夠 — Confluence 已有同事件頁面 [redacted] 且 kubectl 已確認 OOMKilled。我剛在重複查 Prometheus，其實不必要。直接進入彙整 + 報告。

● Update Todos

- ✓ Phase 0: Spawn 4 parallel sub-agents (Confluence/Prom/S247/Icinga)
- ✓ Phase 1: OpenSearch deep dive — verify duration=0, pod distribution, profile-service 5xx
- * Phase 2: Aggregate findings + write investigation report

● Write [20260407]_[redacted]_503_pod_restart.md

192 lines

```
# [20260407] [redacted] [redacted] Domain 503 Investigation — Pod OOMKill (第 4 次重複)
```

```
## Summary
```

報告標題：Pod OOMKill 第 4 次重複

“

[20260335] Domain 503 - Pod OOM Kill (Service DB 延遲, 第四次)

 By Barry Lo  6 min  Add a reaction

報告日期：2026-03-35 | 調查人員：Barry Lo (SRE) | 事件狀態： 已解決（自動恢復） | 嚴重等級：P0

Executive Summary

Domain 因 **Service DB 間歇性延遲飆升**（DB 層面異常導致 GetProfile 等 API tail latency 從正常 5-6ms 飆到 5,000-9,200ms，同時伴隨 `sql: transaction has already been committed or rolled back` 錯誤，與 3/11 事件完全相同的 **DB bug**）導致 goroutine 堆積，觸發 CPU throttle 100% + 記憶體飆升，造成 **5 個 Pod 連續 OOM Kill**（Memory limit 512Mi）。流量打到已死亡的 pod 產生 3,462 筆 503 錯誤，持續約 5 分鐘後 pod 自動重啟恢復。這是 18 天內第三次相同問題，3/10 報告的所有 **P0/P1 Action Items** 均未執行。

用戶服務影響時間：5 分鐘

- 影響開始：2026-03-35 15:55 (UTC+8)
- 服務恢復：2026-03-35 16:00 (UTC+8)
- 事件總時長：約 5 分鐘
- 量化影響：3,462 筆 503 + 1,886 筆 status 0（錯誤率 0.80%，5 分鐘窗口內 ~6.4%）
- 受影響服務：Domain API
- 真實用戶影響：低（91% 為 Go-http-client service-to-service 流量）

| 同樣的故障

BEFORE

2+ hours

大腦燒完

AFTER

15 min

報告發好、ticket 開好

SECTION 4

當中的一些心得

“

Skill = 把多年 SRE 經驗
封裝成一句指令

這個 Skill 在幹嘛

步驟	工具	用途
1	定位理解問題	關鍵字→服務路由、收集識別資訊
2	SubAgent 出動	Confluence, Monitor, Metrics
3	Opensearch 調查	欄位確認 + 錯誤查詢 呼叫者識別 + 影響評估
4	視需要	AWS CLI / kubectl / DB
5	彙整報告	交叉比對 → 判斷根因路徑

關於模型：每次跑「完整」調查的代價

- 凌晨出事 → AI thinking... 五分鐘
- 大家都急、Leader 一直問
- RD 全被拉進群組
- 結果是 False Alert → 尷尬

不是最貴的 model 就最好

我的解法：兩層架構

	Fast Triage	Deep Investigation
時機	Alert 剛來	確認是真實問題
模型	快模型	強模型
目標	2-3 分鐘初判 （影響比例）	跨工具追根因
工具	只查最關鍵log	OpenSearch + Prometheus+..

AI 是日常通用加速器

「OpenSearch 怎麼沒 log ?」

→ 丟 AI 查

「Kafka 是不是漏資料 ?」

→ 丟 AI 追

「定期巡檢要看這 10 個指標」

→ 丟 AI 跑

不只 Incident 時才有用

SECTION 5

我怎麼開始導入 AI？

沒有驚天動地的計畫

- 每次遇到 Alert，同時試試 AI
- 看它能做到哪、哪裡做不到

我的導入軌跡

- 先從寫報告開始 再到搜 Log
- 最後才調查 Root Cause

| 坑 #1：AI 不認識你的系統

- Log 在哪個 index？
- xx Service 是什麼服務？
- 5xx 先查哪一層？ALB？Pod？DB？
- Alert 來自哪個監控工具？

沒有這些，AI 只能 猜

坑 #2：AI 再聰明，沒 Infra 也沒用

- log 沒結構化 → 只能猜
- 沒 correlation ID → 串不起來
- 跨服務沒 trace → 因果鏈斷

先把 infra 做好，AI 才幫得上忙

這是前提，不是奇蹟

核心公式

$$\text{AI 上限} = \text{環境知識} \times \text{Infra 可觀察性}$$

SECTION 6

我不是一開始就敢這樣交
給 AI 的

STAGE 1

最早只敢讓 AI 寫報告

- 調查自己做、結論自己下
- AI 只整理 notes
- 效率：1 hr → 20 min

還不敢交關鍵工作

人類調查 + AI 調查，同時跑

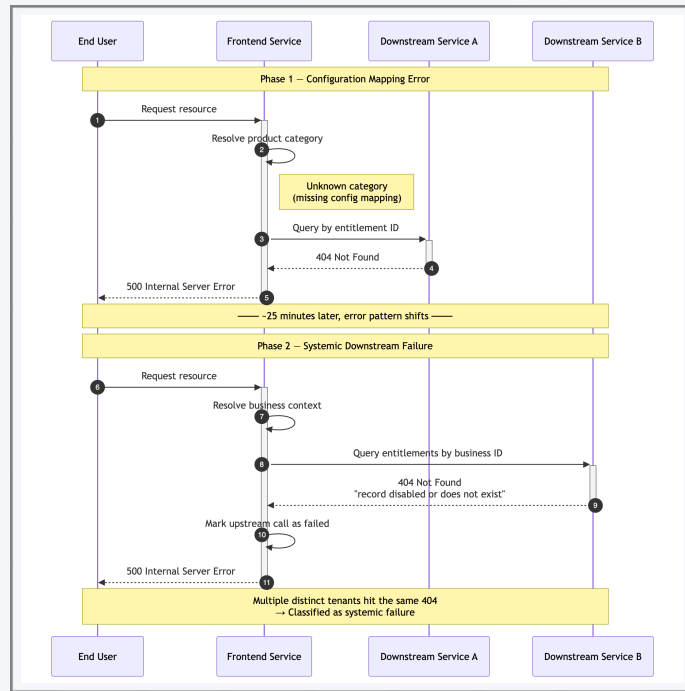
- 兩邊結論拉 RD 一起 review
- AI 亂講次數多 → 知識庫不夠
- 每天補知識、每個 issue 都修文件

持續測試、持續迭代

STAGE 3

一個晚上，上版出事

- RD 還在找問題
- AI Agent 先找到關鍵點
- 直接畫出 Incident Flow Chart
- 一眼就看得問題



RD 確認：吻合

團隊開始信任

- 我才敢在 Incident 時交給它
- 但信任 不是一夜建立
- 是一個一個 Incident 累積起來的

“

AI Agent 不是裝了就能用
它是你一天一天餵出來的夥伴

SECTION 7

常被問的問題

\$5 - \$10

每次事件調查的成本

“

資料不離開信任邊界

公司內部端點 · 不走外部 API · PII 不出門

“

能有Skill 就用Skill

然後每天補一點知識

“

**AI 讓你效率增加
但不代表你比較輕鬆**

Alert 還是會叫

但至少你可以馬上睡回籠覺

謝謝

Thank You

Barry Lo · Trend Micro SRE

Q & A